Stats 21: Homework 1

Ayla Lee

Acknowledgements: several of these problems are copied from or modified from Think Python by Allen Downey.

I've started the homework file for you. You'll need to fill in the rest with your answers.

My encouragement is to use the keyboard shortcuts as much as possible and use the mouse as little as possible while working the Jupyter Notebook.

After you complete the homework with your answers, in the toolbar, select "Clear All Outputs" and then click "Run All".

Review the document to **make sure all requested output is visible**. You will not get credit for problems where the requested output is not visible, even if the function you coded is correct.

When you are satisfied with the output, click the `...` (to the right of Outline), select "Export", and then Export to HTML.Finally, open the HTML file and "Print as PDF". Submit the PDF to Gradescope.

Submit this ipynb file, complete with your answers and output to Canvas / Bruin Learn.

Again, you must make sure all requested output is visible to receive full credit.

Task 1

Create an account on GitHub.

Change your profile picture. Ideally, use photo of yourself that would be appropriate for a resume. If you are not comfortable with the idea of using a photo of yourself, use any other image that is suitable for a workplace environment.

Follow the instructions provided in class to fork the class repository to your GitHub.

Create another repository with at least two text files in it on GitHub (other than the forked class notes repository). Make at least two additional commits to the reposiotry and push them to GitHub.

Provide a link to both repositories here.

You will also need to submit the link to your own repository (not the forked one) to Canvas / Bruin Learn.

Your Answer:

- Link to your forked repository: https://github.com/ayla2514/26-spring-stats-21.git
- Link to your own repository: https://github.com/ayla2514/stats21-hw1-practice.git

Problem 2

An important part of programming is learning to interpret error messages and understanding what correction needs to be made.

Read and familiarize yourself with the following error messages.

Explain the error. Then duplicate each cell and correct the error. The first problem has been done for you as an example.

In [10]:
```
# A
print("Hello World"
```

```
  Cell In[10], line 2
    print("Hello World"
                       ^
_IncompleteInputError: incomplete input
```

Answer: The `print()` function is missing the closing parenthesis. This results in an unexpected EOF error.

In [11]:
```
# corrected:
print("Hello World")
```

```
Hello World
```

In [12]:
```
# B
print("Hello")
    print("Goodbye")
```

```
  Cell In[12], line 3
    print("Goodbye")
    ^
IndentationError: unexpected indent
```

Answer: The indent should be deleted.

In [13]:
```
# corrected:
print("Hello")
print("Goodbye")
```

```
Hello
Goodbye
```

In [14]:
```
# C
x = 10
if x > 8
    print("x is greater than 8")
```

```
  Cell In[14], line 3
    if x > 8
            ^
SyntaxError: expected ':'
```

Answer: The error caused by missing colon(:)

In [15]:
```
# corrected:
x = 10
if x > 8:
    print("x is greater than 8")
```

```
x is greater than 8
```

In [16]:
```
# D
if x = 10:
    print("x is equal to 10")
```

```
  Cell In[16], line 2
    if x = 10:
       ^
SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?
```

Answer: Since condition should be true or false, = replaced by ==.

In [17]:
```
# corrected:
if x == 10:
    print("x is equal to 10")
```

```
x is equal to 10
```

In [18]:
```
# E
x = 5
if x == 5:
print("x is five")
```

```
  Cell In[18], line 4
    print("x is five")
    ^
IndentationError: expected an indented block after 'if' statement on line 3
```

Answer: Indent is required

In [19]:
```
# corrected:
x = 5
if x == 5:
    print("x is five")
```

```
x is five
```

In [20]:
```
# F
l = [1, 2, 50, 10]
l = sort(l)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[20], line 3
      1 # F
      2 l = [1, 2, 50, 10]
----> 3 l = sort(l)

NameError: name 'sort' is not defined
```

Answer: Use .sort() to sort the list in-place

In [21]:
```
#corrected:
l = [1, 2, 50, 10]
l.sort()
print(l)
```

```
[1, 2, 10, 50]
```

Problem 3

Use Python as a calculator. Enter the appropriate calculation in a cell and be sure the output value is visible.

A. How many seconds are there in 42 minutes 42 seconds?

In [22]:
```
42 * 60
```

Out[22]:
```
2520
```

B. There are 1.61 kilometers in a mile. How many miles are there in 10 kilometers?

In [23]:
```
10 / 1.61
```

Out[23]:
```
6.211180124223602
```

C. If you run a 10 kilometer race in 42 minutes 42 seconds, what is your average 1-mile pace (time to complete 1 mile in minutes and seconds)? What is your average speed in miles per hour?

In [24]:
```
dist_km = 10
min_val = 42
sec_val = 42

miles = dist_km / 1.61
total_seconds = (min_val * 60) + sec_val
```

```
#Pace (time per mile)
pace_sec_per_mile = total_seconds / miles
pace_min = int(pace_sec_per_mile // 60)
pace_sec = int(pace_sec_per_mile % 60)

# Calculate Speed (miles per hour)
hours = total_seconds / 3600
speed_mph = miles / hours


print(f"Pace: {pace_min} min {pace_sec} sec per mile")
print(f"Speed: {speed_mph:.2f} mph")
```

```
Pace: 6 min 52 sec per mile
Speed: 8.73 mph
```

Problem 4

Write functions for the following problems.

A. The volume of a sphere with radius r is

$$V = \frac{4}{3}\pi r^3$$

Write a function `sphere_volume(r)` that will accept a radius as an argument and return the volume.

- Use the function to find the volume of a sphere with radius 5.
- Use the function to find the volume of a sphere with radius 15.

In [25]:
```
import math

# Define the function
def sphere_volume(r):
    """
    Calculate the volume of a sphere with radius r.
    """
    volume = (4/3) * math.pi * (r**3)
    return volume

# 1. Volume with radius 5
vol_5 = sphere_volume(5)
print(f"Volume with radius 5: {vol_5:.2f}")

# 2. Volume with radius 15
vol_15 = sphere_volume(15)
print(f"Volume with radius 15: {vol_15:.2f}")
```

```
Volume with radius 5: 523.60
Volume with radius 15: 14137.17
```

B. Suppose the cover price of a book is $24.95, but bookstores get a 40% discount. Shipping costs $3 for the first copy and 75 cents for each additional copy.

Write a function `wholesale_cost(books)` that accepts an argument for the number of books and will return the total cost of the books plus shipping.

- Use the function to find the total wholesale cost for 60 copies.
- Use the function to find the total wholesale cost for 10 copies.

In [26]:
```
def wholesale_cost(books):

    cover_price = 24.95
    discounted_price = cover_price * 0.60  # 40% discount means paying 60%

    total_book_price = discounted_price * books

    # $3 for the first copy, $0.75 for each additional copy
    if books > 0:
        shipping = 3.00 + (books - 1) * 0.75
    else:
        shipping = 0.00

    total_cost = total_book_price + shipping
    return total_cost

cost_60 = wholesale_cost(60)
print(f"Total cost for 60 copies: ${cost_60:.2f}")

cost_10 = wholesale_cost(10)
print(f"Total cost for 10 copies: ${cost_10:.2f}")
```

```
Total cost for 60 copies: $945.45
Total cost for 10 copies: $159.45
```

C. A person runs several miles. The first and last miles are run at an 'easy' pace. Other than the first and last miles, the other miles are at a faster pace.

Write a function `run_time(miles, warm_pace, fast_pace)` to calculate the time the runner will take. The function accepts three input arguments: how many miles the runner travels (minimum value is 2), the warm-up and cool-down pace, the fast pace. The function will print the time in the format minutes:seconds, and will return a tuple of values: (minutes, seconds)

Use the function to find the time to run a total of 5 miles. The warm-up pace is 8:15 per mile. The speed pace is 7:12 per mile.

Call the function using: `run_time(miles = 5, warm_pace = 495, fast_pace = 432)`

In [27]:
```
def run_time(miles, warm_pace, fast_pace):
```

```
    warm_miles = 2
    fast_miles = miles - 2

    total_seconds = (warm_miles * warm_pace) + (fast_miles * fast_pace)

    # Convert to minutes and seconds
    minutes = int(total_seconds // 60)
    seconds = int(total_seconds % 60)

    print(f"{minutes}:{seconds:02d}")

    return (minutes, seconds)

# Call the function for 5 miles
# warm_pace = 495 seconds (8:15), fast_pace = 432 seconds (7:12)
result = run_time(miles = 5, warm_pace = 495, fast_pace = 432)
```

```
38:06
```

Another important skill is to be able to read documentation.

Read the documentation for the function str.split() at
https://docs.python.org/3/library/stdtypes.html#str.split

Adjust the function so that the call can be made with minutes and seconds:

`run_time(miles = 5, warm_pace = "8:15", fast_pace = "7:12")`

In [28]:
```
def run_time(miles, warm_pace, fast_pace):

    # Helper logic to convert "MM:SS" string to total seconds
    def to_seconds(pace_str):
        # split(":") separates the string into a list
        parts = pace_str.split(":")
        return int(parts[0]) * 60 + int(parts[1])

    # Convert string paces to integer seconds
    warm_sec = to_seconds(warm_pace)
    fast_sec = to_seconds(fast_pace)

    total_seconds = (2 * warm_sec) + ((miles - 2) * fast_sec)

    # Convert back to minutes and seconds for output
    minutes = int(total_seconds // 60)
    seconds = int(total_seconds % 60)

    print(f"{minutes}:{seconds:02d}")
    return (minutes, seconds)

# Call the function with string arguments
result = run_time(miles = 5, warm_pace = "8:15", fast_pace = "7:12")
```

```
38:06
```

Problem 5

Use `import math` to gain access to the math library.

Create a function `polar(real, imaginary)` that will return the polar coordinates of a complex number.

The input arguments are the real and imaginary components of a complex number. The function will return a tuple of values: the value of the radius `r` and the angle `theta`.

For a refresher, see: https://ptolemy.berkeley.edu/eecs20/sidebars/complex/polar.html

Show the results for the following complex numbers:

- 1 + i
- -2 - 3i
- 4 + 2i

In [29]:
```
import math

def polar(real, imaginary):
    # Calculate radius r using Pythagorean theorem
    r = math.sqrt(real**2 + imaginary**2)

    # Calculate angle theta in radians using atan2
    theta = math.atan2(imaginary, real)

    return (r, theta)

# Test cases
test_cases = [(1, 1), (-2, -3), (4, 2)]

for real, imag in test_cases:
    r, theta = polar(real, imag)
    print(f"Input: {real} + {imag}i -> Radius: {r:.4f}, Theta: {theta:.4f}")
```

```
Input: 1 + 1i -> Radius: 1.4142, Theta: 0.7854
Input: -2 + -3i -> Radius: 3.6056, Theta: -2.1588
Input: 4 + 2i -> Radius: 4.4721, Theta: 0.4636
```

Problem 6

Define a function called `insert_into(listname, index, iterable)`. It will accept three arguments, a currently existing list, an index, and another list/tuple that will be inserted at the index position.

Python's built-in function, `list.insert()` can only insert one object.

In [30]:
```
# write your code here

def insert_into(listname, index, iterable):
    # Use list slicing to insert an iterable at a specific index
    listname[index:index] = iterable
    return listname

# Test the function
my_list = [1, 2, 5, 6]
new_items = [3, 4]

# Insert [3, 4] at index 2
result = insert_into(my_list, 2, new_items)
print(f"Resulting list: {result}")
```

```
Resulting list: [1, 2, 3, 4, 5, 6]
```

In [31]:
```
# do not modify. We will check this result for grading
l = [0,'a','b','c',4,5,6]
i = ['hello', 'there']
insert_into(l, 3, i)
```

Out[31]:
```
[0, 'a', 'b', 'hello', 'there', 'c', 4, 5, 6]
```

Problem 7

Define a function called `first_equals_last(listname)`

It will accept a list as an argument. It will return `True` if the first and last elements are equal and the if the list has a length greater than 1. It will return False for all other cases.

In [32]:
```
# write your function here
def first_equals_last(listname):
    # Condition 1: Length is greater than 1
    # Condition 2: First element (index 0) equals last element (index -1)
    return len(listname) > 1 and listname[0] == listname[-1]
```

In [33]:
```
# do not modify. We will check this result for grading
a = [1,2,3]
first_equals_last(a)
```

Out[33]:
```
False
```

In [34]:
```
# do not modify. We will check this result for grading
b = ['hello','goodbye','hello']
first_equals_last(b)
```

Out[34]:
```
True
```

In [35]:
```
# do not modify. We will check this result for grading
c  = [1,2,3,'1']
```

```
first_equals_last(c)
```

Out[35]: False

In [36]:
```
# do not modify. We will check this result for grading
d = [[1,2],[3,2],[1,2]]
first_equals_last(d)
```

Out[36]: True